Bilan complet de la séance Gastronome Créatif

Dimanche 11 octobre 2026 | ASP.NET Core MVC · C# · Razor · Entity Framework Core · PostgreSQL

Aujourd'hui, nous avons finalisé la partie fonctionnelle de la section « Recettes à la une » et développé une recherche dynamique de recettes, depuis le formulaire HTML jusqu'aux fiches détaillées. Les compilations et les principaux tests de navigation ont été validés.

1. Finalisation de « Recettes à la une »

1.1. Récupération et affichage des recettes

Dans HomeController.cs, nous avons utilisé le contexte Entity Framework Core pour récupérer trois recettes de PostgreSQL, puis transmettre la liste à la vue :

```
public IActionResult Index()
{
    List<Recettes> recettes = _context.Recettes.Take(3).ToList();
    return View(recettes);
}
```

La vue Views/Home/Index.cshtml déclare @model List<GC_MVC.Models.Recettes> et parcourt cette liste avec foreach. Les recettes observées lors des tests étaient : Saucisse de Morteau à l'aubergine ; Saumon à la plancha, crème au wasabi ; Pizza à la pastèque.

1.2. Images dynamiques

La colonne photo contient le nom de fichier. Les images des trois recettes testées sont placées dans wwwroot/img/illustrations/. Nous avons donc construit leur chemin avec Razor :

```
<img src="~/img/illustrations/@recettes.photo"
     alt="@recettes.nom" />
```

1.3. Résolution du problème CSS

Les images étaient surdimensionnées. Le fichier styleHome.css comportait deux sélecteurs concurrents : l'un définissait width: 100% et était plus spécifique que celui imposant 150px. Nous avons simplifié les règles, organisé les cartes avec Flexbox et stabilisé la taille des images.

```
.recette .section-recette {
    display: flex;
    justify-content: space-around;
    align-items: stretch;
    gap: 20px;
    padding: 20px;
}
.recette .section-recette .recette1 {
    display: flex;
    flex: 1;
    align-items: center;
    gap: 15px;
    min-width: 0;
}
.recette .section-recette .recette1 img {
```

```
    width: 150px;
    height: 200px;
    object-fit: cover;
    flex-shrink: 0;
}
```

Le résultat a été validé : les images et les titres sont alignés. Les styles des notes et du bouton « Voir nos autres recettes gourmandes » ont été conservés.

1.4. Liens vers les fiches détaillées

Nous avons utilisé les Tag Helpers asp-controller, asp-action et asp-route-id pour lier chaque titre, puis chaque image, à sa fiche détaillée :

```
<a asp-controller="Recettes"
   asp-action="Detail"
   asp-route-id="@recettes.id_recette">
    @recettes.nom
</a>
```

Test validé : cliquer sur le titre ou l'image ouvre la fiche correspondant à la recette sélectionnée. La fonctionnalité est terminée ; des finitions graphiques restent possibles.

2. Développement de la recherche de recettes

2.1. Formulaire HTML avec la méthode GET

La page Views/Recherche/Index.cshtml possédait déjà son interface. Nous avons remplacé le conteneur de recherche par un formulaire GET, ajouté name="terme" au champ et transformé le bouton en type="submit".

```
<form class="recherche" asp-controller="Recherche"
      asp-action="Index" method="get">
    <input type="text" id="champRecherche" name="terme"
           placeholder="Rechercher une recette, un ingrédient..." />
    <button type="submit" id="boutonRecherche"
            aria-label="Rechercher">...</button>
</form>
```

Test validé : la recherche de « chocolat » produit une URL de type /Recherche?terme=chocolat.

2.2. Model binding : récupération du paramètre

L'action du contrôleur a été modifiée pour accepter string? terme. ASP.NET Core MVC associe automatiquement le paramètre d'URL au paramètre de méthode.

```
public IActionResult Index(string? terme)
{
    Console.WriteLine($"Terme recherché : {terme}");
    return View();
}
```

Nous avons distingué la console du navigateur du terminal ASP.NET Core. Le terminal a bien affiché « Terme recherché : chocolat », confirmant la récupération de la valeur.

2.3. Injection de GCContext

Nous avons ajouté les espaces de noms GC_MVC.Data et GC_MVC.Models, puis injecté GCContext dans RechercheController pour accéder à PostgreSQL.

```
private readonly GCContext _context;

public RechercheController(GCContext context)
{
    _context = context;
}
```

La compilation avec dotnet build a réussi.

2.4. Première requête LINQ

Tu as écrit toi-même la requête de filtrage par nom :

```
List<Recettes> recettes = _context.Recettes
    .Where(r => r.nom.Contains(terme))
    .ToList();
```

Where() filtre les recettes ; r => ... est une expression lambda ; Contains(terme) vérifie la présence du texte ; ToList() exécute la requête et matérialise la liste. Le filtrage ne classe pas automatiquement les résultats par pertinence.

2.5. Correction de l’erreur CS0103

Une variable recettes avait été déclarée dans un bloc if puis utilisée après ses accolades. Le compilateur a signalé : CS0103 : Le nom « recettes » n’existe pas dans le contexte actuel. Nous avons étudié la portée des variables et déplacé la déclaration avant le if.

```
List<Recettes> recettes = new List<Recettes>();

if (!string.IsNullOrWhiteSpace(terme))
{
    recettes = _context.Recettes
        .Where(r => r.nom.Contains(terme))
        .ToList();
}

return View(recettes);
```

La nouvelle compilation a réussi. Nous avons ainsi distingué déclaration et réaffectation d’une variable.

2.6. Affichage des résultats dans Razor

La vue a été typée avec @model List<GC_MVC.Models.Recettes>, puis une boucle foreach a permis d’afficher les noms des recettes trouvées. La recherche de « chocolat » a affiché « Mousse au chocolat ».

```
@foreach (var recette in Model)
{
    <div class="resultat-recette">
        <h2>@recette.nom</h2>
    </div>
}
```

2.7. Recherche insensible à la casse

Nous avons ajouté using Microsoft.EntityFrameworkCore; et remplacé Contains() par EF.Functions.ILike(), adapté à PostgreSQL. Les caractères % permettent de rechercher le terme n’importe où dans le nom.

```
.Where(r => EF.Functions.ILike(r.nom, $"%{terme}%"))
```

Test validé : « chocolat », « Chocolat » et « CHOCOLAT » retournent les mêmes résultats dans nos essais.

2.8. Gestion des recherches sans résultat

Nous avons ajouté une condition sur Model.Count :

```
@if (Model.Count == 0)
{
    <p>Aucune recette trouvée.</p>
}
else
{
    @foreach (var recette in Model)
    {
        <div class="resultat-recette">
            <h2>@recette.nom</h2>
        </div>
    }
}
```

Deux tests ont réussi : « chocolat » affiche la recette correspondante ; « xyz123aucunerecette » affiche « Aucune recette trouvée. ». Nous avons clarifié que le titre HTML fixe « Résultats de la recherche » reste affiché indépendamment des résultats.

2.9. Navigation des résultats vers les détails

Tu as ajouté les Tag Helpers dans la boucle de recherche. Nous avons corrigé une balise </h2> incomplète et un chevron supplémentaire après .

```
@foreach (GC_MVC.Models.Recettes recette in Model)
{
    <div class="resultat-recette">
        <a asp-controller="Recettes" asp-action="Detail"
           asp-route-id="@recette.id_recette">
            <h2>@recette.nom</h2>
        </a>
    </div>
}
```

Test validé : cliquer sur « Mousse au chocolat » depuis les résultats ouvre sa fiche détaillée.

3. État récapitulatif du RechercheController

Le code ci-dessous reconstitue l'état logique validé durant la séance ; il ne constitue pas une nouvelle inspection du fichier local après tes dernières modifications.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using GC_MVC.Data;
using GC_MVC.Models;

namespace GC_MVC.Controllers;

public class RechercheController : Controller
{
    // Contexte d'accès à PostgreSQL.
    private readonly GCContext _context;

    // Injection de dépendances.
    public RechercheController(GCContext context)
    {
        _context = context;
    }

    // Récupère le terme envoyé par le formulaire GET.
```

```
    public IActionResult Index(string? terme)
    {
        // Liste vide par défaut.
        List<Recettes> recettes = new List<Recettes>();

        // Recherche uniquement si un terme a été saisi.
        if (!string.IsNullOrWhiteSpace(terme))
        {
            recettes = _context.Recettes
                .Where(r => EF.Functions.ILike(r.nom, $"%{terme}%"))
                .ToList();
        }

        // Transmission des résultats à Razor.
        return View(recettes);
    }
}
```

4. Notions techniques consolidées

Notion	Compétence pratiquée
ASP.NET Core MVC	Faire communiquer formulaire, contrôleur et vue
Formulaire GET / model binding	Transmettre et récupérer un paramètre URL
Injection de dépendances	Utiliser GCContext dans un contrôleur
LINQ / expressions lambda	Filtrer des recettes avec Where()
Entity Framework Core	Interroger PostgreSQL depuis C#
ILike()	Rechercher sans distinguer majuscules et minuscules
ToList()	Exécuter et récupérer les résultats
Portée des variables	Déclarer et réaffecter dans le bon bloc
Razor / foreach / if	Afficher une liste et gérer les résultats vides
Tag Helpers	Relier les résultats aux fiches recettes
CSS / Flexbox	Corriger les priorités CSS et l’alignement
Débogage	Interpréter CS0103 et vérifier les logs

5. Points à reprendre à la prochaine séance

• Ajouter les images aux résultats de recherche.

• Distinguer une page ouverte sans recherche d’une recherche réellement effectuée mais sans résultat.

• Améliorer la présentation et l’affichage responsive de la page de recherche.

• Poursuivre les fonctionnalités prioritaires : inscription, connexion, gestion des recettes et droits d’administration.

• Tester le site sur mobile et tablette ; finaliser les détails visuels et le favicon.

6. Planning et conclusion

L’objectif reste une version fonctionnelle du projet au 30 octobre 2026, afin de consacrer la période suivante au rapport, au diaporama et aux vérifications avant l’échéance du 12 novembre 2026. La priorité est donc donnée aux fonctionnalités, les finitions graphiques venant ensuite.

Acquis essentiel de la journée : tu sais désormais faire circuler une donnée depuis un formulaire HTML vers un contrôleur MVC, l’utiliser pour filtrer des recettes dans PostgreSQL avec Entity Framework Core, afficher les résultats dans Razor et naviguer jusqu’à la fiche détaillée. Pense à enregistrer ton travail et à effectuer un commit Git.